Système de gestion d’une résidence universitaire

Rapport de modélisation UML — novembre 2026

1. Contexte et fonctionnalités

Une résidence universitaire doit gérer ses chambres, ses occupants, les demandes d’hébergement et les périodes d’occupation. Le responsable souhaite disposer d’une application pour suivre les attributions de chambres et éviter les affectations multiples.

Fonctionnalités attendues

- Enregistrer les demandes d’hébergement.
- Attribuer une chambre à un étudiant.
- Vérifier les places disponibles.
- Enregistrer les départs.
- Consulter les chambres occupées et libres.
- Consulter l’historique des occupations.
- Extension : calculer le taux d’occupation de la résidence et afficher des statistiques.

Modélisation UML réalisée

Diagramme	Objectif
Diagramme de cas d’utilisation	Délimiter les fonctions du système et identifier ses acteurs.
Diagramme de classes	Décrire la structure des données et les relations entre les classes.
Diagramme de séquence	Détailler, étape par étape, l’attribution d’une chambre.
Diagramme d’états-transitions	Suivre le cycle de vie d’une demande d’hébergement.

Classes retenues

`Etudiant`, `Residence`, `Batiment`, `Chambre`, `DemandeHebergement` et `Attribution`.

Règle de gestion centrale : une chambre ne peut pas recevoir plus d’occupants en cours que sa capacité. C’est cette règle, contrôlée avant chaque attribution, qui évite les affectations multiples.

2. Diagramme de cas d’utilisation

Deux acteurs interagissent avec le système : le responsable de la résidence, qui pilote la gestion, et l’étudiant, qui dépose sa demande d’hébergement. « Vérifier les places disponibles » est inclus dans l’attribution, et l’affichage des statistiques est une extension.

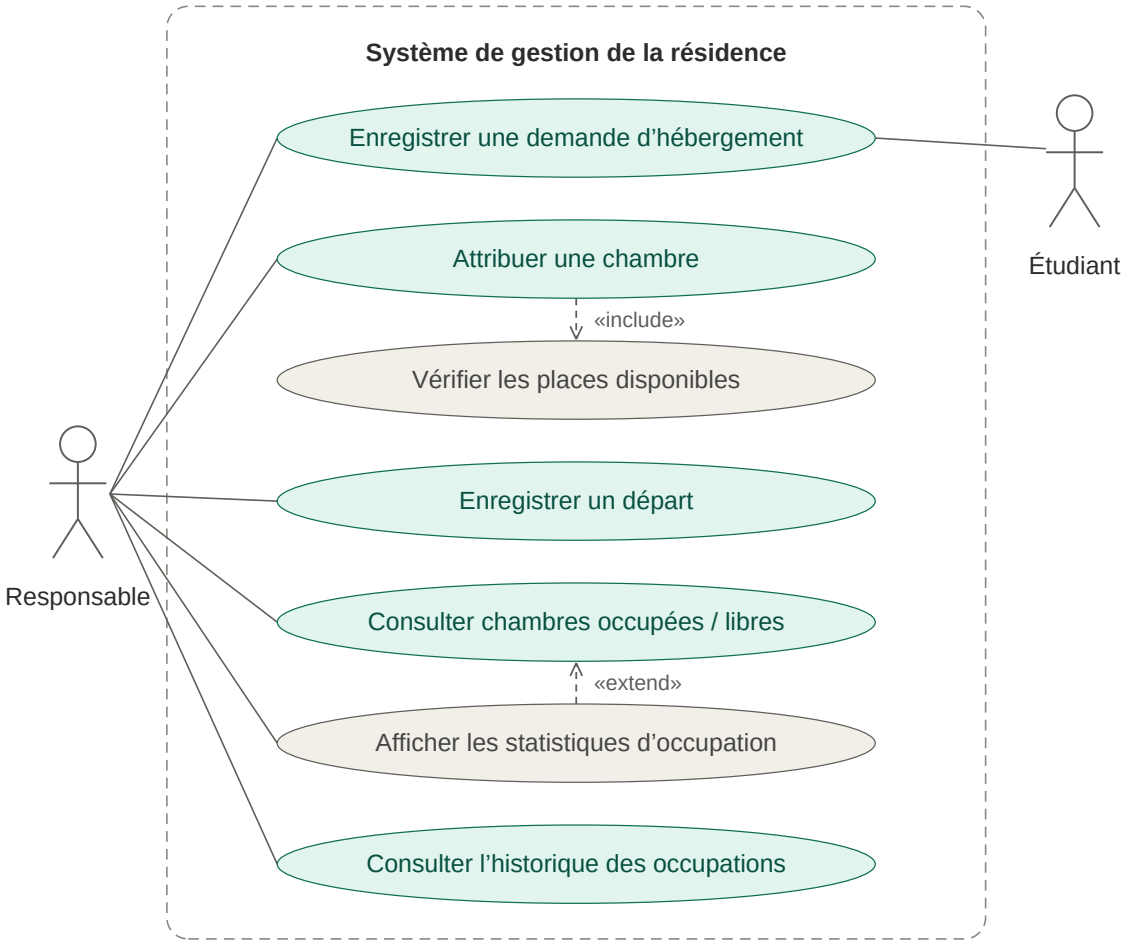

Figure 1 — Cas d’utilisation du système de gestion de la résidence.

Cas d’utilisation	Acteur	Rôle
Enregistrer une demande d’hébergement	Étudiant, responsable	Saisir la demande et les dates souhaitées.
Attribuer une chambre	Responsable	Affecter une chambre à une demande validée.
Vérifier les places disponibles	Inclus dans l’attribution	Contrôler qu’une place est libre.
Enregistrer un départ	Responsable	Clôturer l’occupation et libérer la place.
Consulter chambres occupées / libres	Responsable	Voir l’état courant de la résidence.
Afficher les statistiques d’occupation	Responsable (extension)	Taux d’occupation et indicateurs.
Consulter l’historique des occupations	Responsable	Occupations passées par chambre ou étudiant.

3. Diagramme de classes

Le modèle sépare la structure physique de la résidence (`Residence`, `Batiment`, `Chambre`) du processus d’hébergement (`Etudiant`, `DemandeHebergement`, `Attribution`). L’attribution relie une demande à une chambre et porte les dates d’occupation, ce qui fournit l’historique.

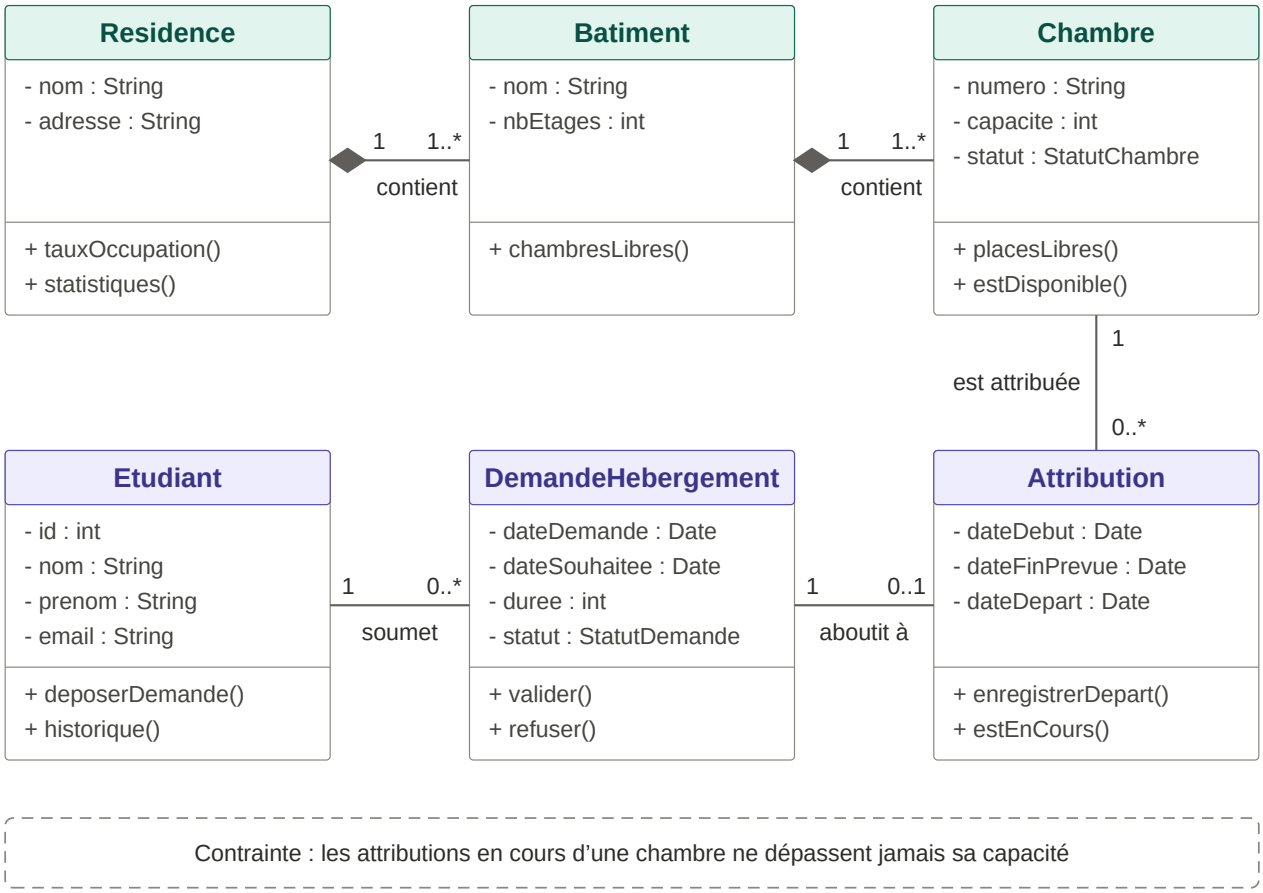

Figure 2 — Diagramme de classes (compositions pleines, associations simples).

Classe	Rôle
Residence	Ensemble des bâtiments ; calcule le taux d’occupation et les statistiques.
Batiment	Regroupe les chambres d’un même bâtiment.
Chambre	Possède une capacité et un statut ; indique ses places libres.
Etudiant	Personne qui dépose des demandes et occupe une chambre.
DemandeHebergement	Demande d’un étudiant, avec son statut (voir diagramme d’états).
Attribution	Lien demande–chambre avec dates de début, de fin prévue et de départ.

4. Diagramme de séquence : attribution d’une chambre

Le responsable demande l’attribution ; le système contrôle la demande et la chambre avant de créer l’attribution. Ce contrôle empêche les affectations multiples.

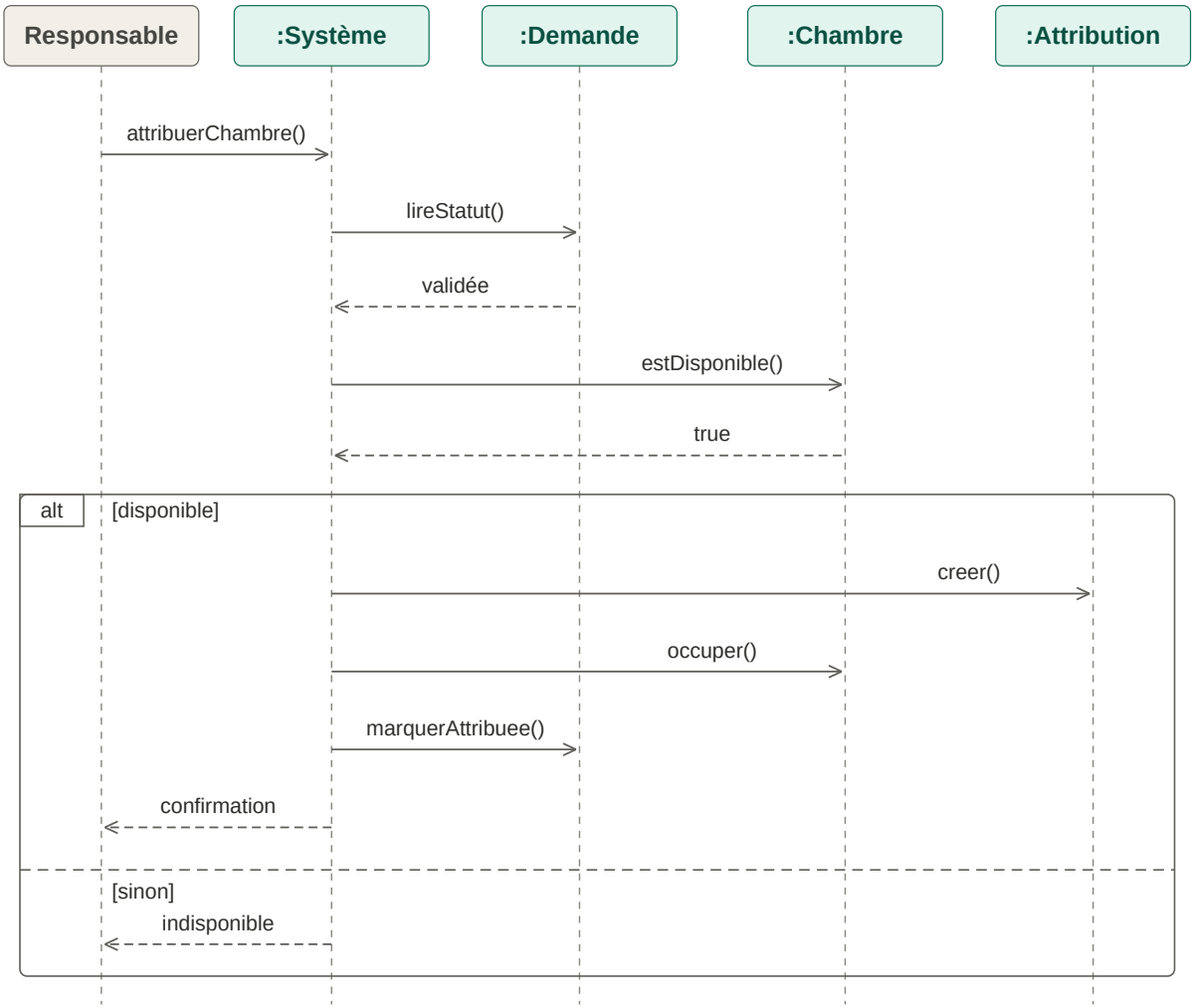

Figure 3 — Séquence de l’attribution d’une chambre (fragment alt : disponible / sinon).

- Le responsable lance `attribuerChambre()` pour une demande et une chambre.
- Le système lit le statut de la demande, qui doit être « validée ».
- Il demande à la chambre si une place est libre (`estDisponible()`).
- Si oui : création de l’attribution, mise à jour de la chambre, demande marquée « Attribuée », confirmation.
- Sinon : le système signale que la chambre est indisponible et ne modifie rien.

5. Diagramme d’états-transitions : demande d’hébergement

Une demande est d’abord enregistrée, puis validée, puis attribuée, et se termine au départ de l’étudiant. Elle peut aussi être refusée ou annulée avant l’attribution.

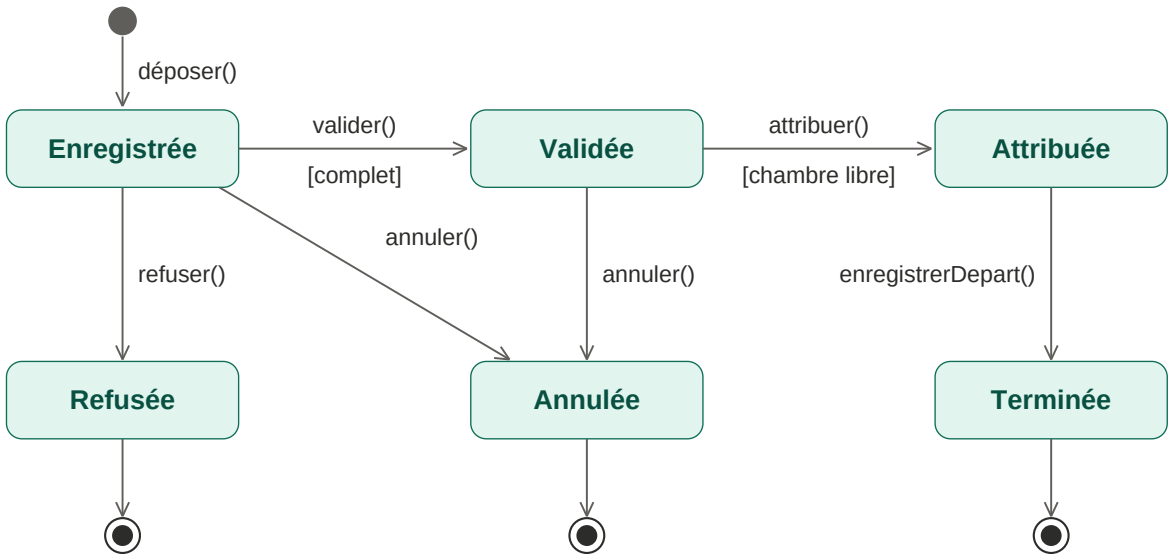

Figure 4 — Cycle de vie d’une demande d’hébergement.

État	Signification	Transitions sortantes
Enregistrée	Demande déposée, en attente de traitement.	valider() [complet], refuser(), annuler()
Validée	Dossier complet, en attente d’une chambre.	attribuer() [chambre libre], annuler()
Attribuée	Chambre affectée, étudiant hébergé.	enregistrerDepart()
Refusée	Demande rejetée (état final).	—
Annulée	Demande annulée avant attribution (état final).	—
Terminée	Départ enregistré, occupation close (état final).	—

6. Extension : taux d’occupation et statistiques

La méthode `Residence.tauxOccupation()` alimente le cas d’utilisation « Afficher les statistiques d’occupation ».

Taux d’occupation = (places occupées ÷ capacité totale) × 100

- Places occupées : nombre d’attributions en cours (sans date de départ).
- Capacité totale : somme des capacités de toutes les chambres de tous les bâtiments.
- Statistiques proposées : taux global, taux par bâtiment, chambres libres, demandes par statut, durée moyenne d’occupation.
- Exemple : 96 places occupées pour 120 places au total donnent un taux de 80 %.

```
tauxOccupation() : float
    total    = somme(chambre.capacite pour chaque chambre)
    occupees = nombre d'attributions en cours
    si total = 0 : retourner 0
    retourner occupees / total * 100
```